IDM → BlackLine Reversal File Code: Process Guide

At a glance

This code takes an incorrect IDM file that was already sent to BlackLine and produces a matching reversal file with every amount sign-flipped (300 becomes -300), so loading it into BlackLine cancels the bad data to zero. It is needed roughly monthly or quarterly, whenever an incorrect file has been processed.

Term	Plain-English meaning
IDM	The system that processes financial data and produces the files sent to BlackLine.
BlackLine	The reconciliation system that receives those files. It cannot delete or roll back processed records.
Erroneous file	An IDM file with wrong data that has already been loaded into BlackLine.
Reversal file	A copy of the erroneous file where every amount has its sign flipped, used to cancel the original.
Net balance	The total after adding the original and reversal amounts. The goal is 0.

Problem statement

When incorrect data is processed in IDM and sent to BlackLine, the amounts in BlackLine become unbalanced and inaccurate. BlackLine has no way to delete or reverse records it has already processed, so the wrong data stays there.

The only fix is to send additional data that offsets the incorrect file. Doing this by hand is slow, repeated every month or quarter, and easy to get wrong, which adds reconciliation risk.

Issue	Impact
Wrong data already sits in BlackLine	Balances are unbalanced and inaccurate
BlackLine cannot delete or roll back	The error cannot be removed, only offset
Manual correction	Extra effort and risk of new reconciliation errors
Recurs monthly or quarterly	The effort is repeated each cycle

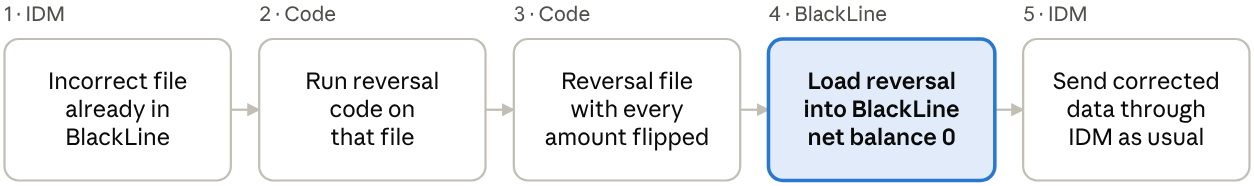
Solution

An AI-assisted Python code generates the reversal file automatically. You give it the erroneous file; it writes a new file with every transaction amount inverted and everything else unchanged.

The idea is simple arithmetic: a transaction of 300 becomes -300, and (+300) + (-300) = 0. Once BlackLine processes the reversal file, the incorrect transactions are offset and the net balance returns to zero.

After the reversal file loads successfully, the corrected source data can be sent and processed normally. This works around BlackLine's inability to delete or roll back processed records.

Loading the reversal file brings BlackLine back to net zero



reversal workflow · 5 steps

The code covers steps 2 and 3; the highlighted step 4 is where the incorrect amounts are offset.

Before the code	With the code
Offsetting data prepared by hand	Reversal file generated automatically

Before the code	With the code
Risk of missed or mistyped amounts	Every amount inverted by the same rule
No built-in check	Original and reversed totals printed for audit

Worked example

The sample IDM_RECURRING file has 25 rows and produces a reversal file with the same 25 rows, where only the Amount column differs. The first six rows show the pattern.

Row	Location	Ledger account	Original Amount	Reversal Amount	Original + Reversal
1	TallahasseeFL	2068002	400	-400.0	0
2	TallahasseeFL	2068005	-400	400.0	0
3	EscondidoCA	2068002	600	-600.0	0
4	EscondidoCA	2068005	-600	600.0	0
5	LawrencevilleGA	2068002	600	-600.0	0
6	LawrencevilleGA	2068005	-600	600.0	0

Across the whole file, the audit totals are:

Check	Original file	Reversal file
Data rows	25	25
Sum of Amount	1,398.00	-1,398.00
Sum of positive amounts	13,455.12	12,057.12
Sum of negative amounts	-12,057.12	-13,455.12

Original plus reversal is 1,398.00 + (-1,398.00) = 0. The original total is not zero because the last row (VanNuysCA, 1,398) has no matching negative line in this sample. The reversal still cancels it exactly.

The reversal file shows amounts as 400.0 rather than 400 because the script converts each value to a decimal number before flipping its sign. The value is the same.

How the script works

The script is Reversal_code.py. It reads the input file once, flips the sign of the amount column, and writes a new file. It never changes the original file.

1. Identify the file type from the file name (table below) to decide which column holds the amount and what separates the columns.
2. Open the file and read the header row. Stop with an error if the file is empty or the amount column is missing.
3. For each data row, read the amount as a number and add it to the original total.
4. Multiply the amount by -1, store it back in the same column, and add it to the modified total.
5. Write all rows to a new file, keeping every other column exactly as it was.
6. Print the audit summary: row count plus original and modified totals.

If the file name contains	Amount column reversed	Column separator
IDM_RECURRING	Amount	Comma (change to tab in the script if the IDM file is tab-separated)
SLTrans	Transaction Amount	Tab
Anything else	None	The script stops with an "Unrecognized input file type" error

The output file is saved next to the input and named by adding _reversal before the extension. For example, IDM_RECURRING_sample_ledger_data_25_rows-1.csv produces IDM_RECURRING_sample_ledger_data_25_rows-1_reversal.csv.

Audit and error handling

After each successful run the script prints an audit summary so a reviewer can confirm the result without opening the files. The modified total must equal the original total with the opposite sign.

Audit line	What it tells you	Sample result
Total rows processed	Number of data rows reversed (header excluded)	25
Original sum of Amount	Net total of the incorrect file	1398.00
Modified sum of Amount	Net total of the reversal file	-1398.00

If anything is wrong the script prints an error and writes no reversal file, so a partial file is never produced by a failed read.

Situation	What the script reports
File is empty or has no header	The input file is empty or has no header
Amount column is missing	Lists the missing column and the columns that were found
An amount is blank or not a number	Names the column, the bad value and the data row number
File cannot be opened or written	Prints the file system error

Advantages

The code removes the manual work of building offsetting data and makes the correction repeatable each month or quarter.

Advantage	What it means
Works around BlackLine's limit	Nothing needs to be deleted or rolled back; the reversal file offsets the wrong data and the net balance returns to 0
Less manual effort	The reversal file is generated automatically from the erroneous file

Advantage	What it means
Consistent and accurate	The same rule (amount × -1) is applied to every row, so there are no hand-typing mistakes
Easy to verify	Row count plus original and modified totals are printed after every run
Original file stays safe	The reversal is written to a new file with _reversal added to the name
Safe failures	Bad input stops the run with a clear message and no partial file is written
Reusable	Handles IDM_RECURRING and SLTrans files and uses only built-in Python modules